COMP2007: Operating Systems & Concurrency
Week 7 – 12:00pm Monday – 04 November 2024

valid for 75 minutes from 11:45am
generated 2024-09-20 14:34

Figure: Attendance Monitoring

Operating Systems and Concurrency

Memory Management 2

COMP2007

Geert De Maere
(Dan Marsden)
{Geert.DeMaere,Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham
United Kingdom

2024

Recall

Last Lecture

- **Mono-programming** with **absolute addressing**
- Modelling **CPU utilisation**
- **Multi-programming** with fixed (non-)equal **partitions** to improve **CPU utilisation**
- **Relocation** and **internal fragmentation**

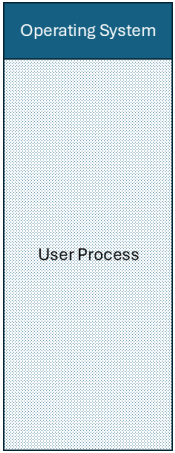


Figure: Summary

Recall

Last Lecture

- **Mono-programming** with **absolute addressing**
- Modelling **CPU utilisation**
- **Multi-programming** with fixed (non-)equal **partitions** to improve **CPU utilisation**
- **Relocation** and **internal fragmentation**

Figure: Summary

Recall

Last Lecture

- **Mono-programming** with **absolute addressing**
- Modelling **CPU utilisation**
- **Multi-programming** with fixed (non-)equal **partitions** to improve **CPU utilisation**
- **Relocation** and **internal fragmentation**

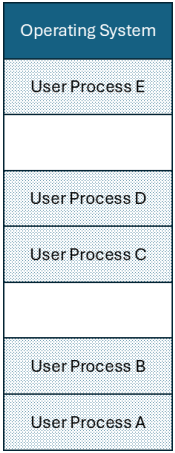


Figure: Summary

Recall

Last Lecture

- **Mono-programming** with **absolute addressing**
- Modelling **CPU utilisation**
- **Multi-programming** with fixed (non-)equal **partitions** to improve **CPU utilisation**
- **Relocation** and **internal fragmentation**

Figure: Summary

Recall

Last Lecture

- **Mono-programming** with **absolute addressing**
- Modelling **CPU utilisation**
- **Multi-programming** with fixed (non-)equal **partitions** to improve **CPU utilisation**
- **Relocation** and **internal fragmentation**

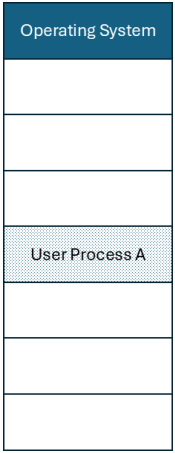


Figure: Summary

Recall

Last Lecture

- **Mono-programming** with **absolute addressing**
- Modelling **CPU utilisation**
- **Multi-programming** with fixed (non-)equal **partitions** to improve **CPU utilisation**
- **Relocation** and **internal fragmentation**

Figure: Summary

Recall

Last Lecture

- **Mono-programming** with **absolute addressing**
- Modelling **CPU utilisation**
- **Multi-programming** with fixed (non-)equal **partitions** to improve **CPU utilisation**
- **Relocation** and **internal fragmentation**

Figure: Summary

Recall

Last Lecture

- **Mono-programming** with **absolute addressing**
- Modelling **CPU utilisation**
- **Multi-programming** with fixed (non-)equal **partitions** to improve **CPU utilisation**
- **Relocation** and **internal fragmentation**

Figure: Summary

Overview

Goals for Today

- Code **relocation** and **protection**
- **Dynamic partitioning** and **swapping**
- **Segmentation**

Relocation and Protection

Example

```
#include <stdio.h>
#include <unistd.h>

int iVar = 0;

void main(int iArgC, char * sArgsv[]) {
    while(iVar < 10) {
      iVar++;
      sleep(2);
      printf("RUN - %s: Address = %p, Value = %d\n",sArgsv[1], &iVar, iVar);
    }
}
```

Relocation and Protection

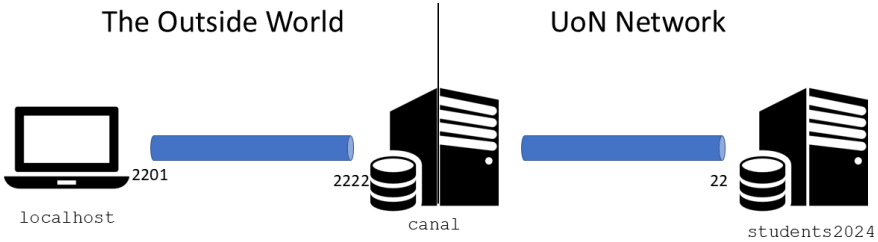
Example

Questions

- Will the value of *iVar* in the first run **influence** the value if *iVar* in the second run[1]?
- Will the **same or different addresses** be displayed for *iVar* if running twice simultaneously?

[1] Note that this may not work on “newer” OSs that use Address Space Layout Randomization

Outside the UoN Network



Relocation and Protection

Principles

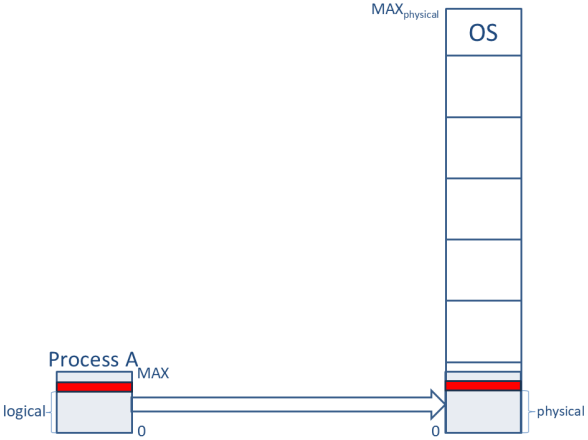


Figure: Address relocation

Relocation and Protection

Principles

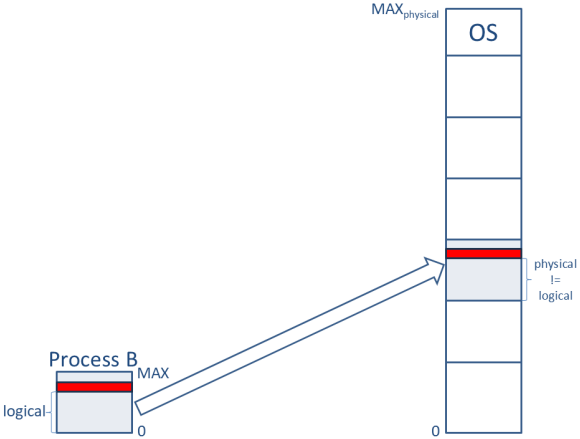


Figure: Address relocation

Relocation and Protection

Principles

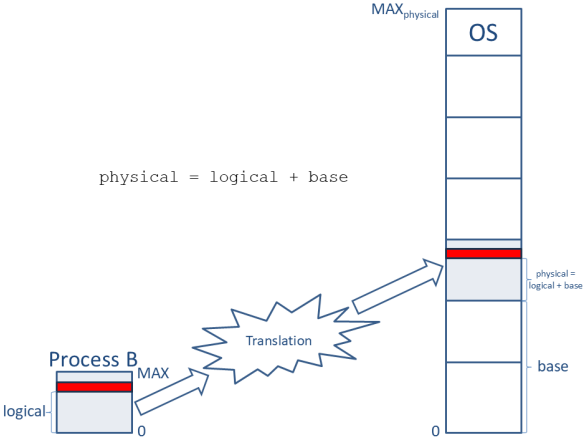


Figure: Address relocation

Relocation and Protection

Example

Solution

- Both processes / runs have the same logical address for `iVar`
- A different offset results in a different physical addresses for `iVar`

```
#include <stdio.h>
#include <unistd.h>

int iVar = 0;

void main(int iArgC, char * sArgsv[]) {
    while(iVar < 10) {
      iVar++;
      sleep(2);
      printf("RUN - %s: Address = %p, Value = %d\n",sArgsv[1], &iVar, iVar);
    }
}
```

Relocation and Protection

Approaches

1. **Static "relocation" at compile time**: a process has to be located at the same location every single time (impractical)
2. **Dynamic relocation** at **load time**
   - An **offset** is added to every logical address to **account for its physical location** in memory
   - **Slows down** the loading of a process
   - Does not account for **swapping**
   - **DLLs** used this approach
3. **Dynamic relocation** at **runtime** with **hardware support**

Relocation and Protection

At Runtime: Base and Bound Registers

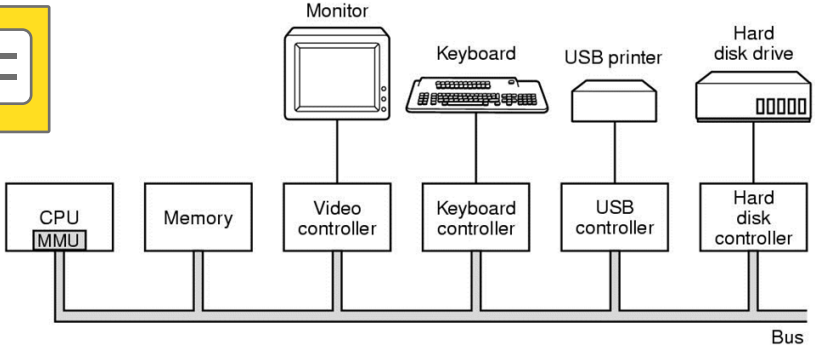


Figure: Simplified computer model (Tanenbaum, 2014)

Relocation and Protection

Principles

- A **logical address** is a memory address **seen by the process**
  - **Relative to the start of the program**
  - Assigned by the **compiler**
  - **Independent** of the current location in **physical memory**
- The **physical address** refers to an **actual location** in **main memory**

Relocation and Protection

Principles

- **Relocation:** one **does not know at compile time** which **partition/addresses** a process will occupy
  - **Compiler** assumes the process that it will run at 0
  - The **logical address space** must be mapped onto the machine's **physical address space** once it is known where in physical memory the process resides
  - **Relocation** must be used in any operating system that allows processes to run at **changing locations** in **physical memory**
- **Protection**: once you can have multiple processes in memory at the same time, **protection** must be enforced

Relocation and Protection

Principles

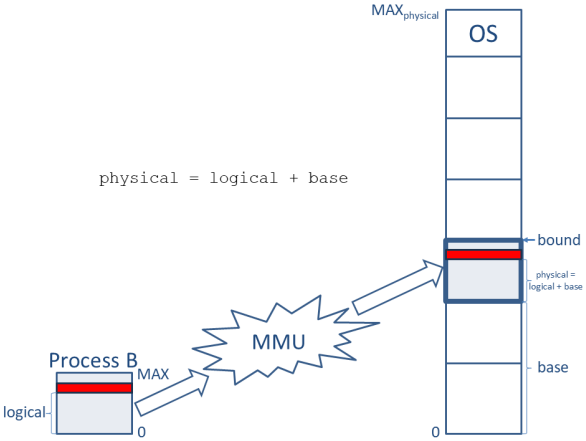


Figure: Address relocation

Relocation and Protection

Base and Bound Registers

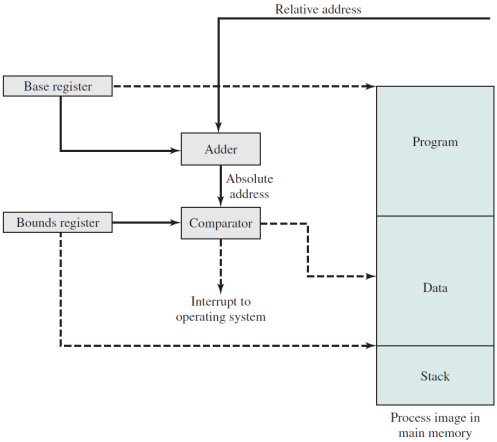


Figure: Address Relocation (Stallings)

Relocation and Protection

At Runtime: Base and Bound Registers

- Two special purpose registers are maintained in the CPU (the **MMU**), containing a **base address** and **bound**
  - The **base register** stores the **start address** of the partition
  - The **bound register** holds the **size** of the partition
- **At runtime**:
  - The **base register** is added to the **logical (relative) address** to generate the **physical address**
  - The resulting address is **compared** against the **bound register**
- **Hardware support** was not always present in the early days!

Dynamic Partitioning

Context

- The **relocation problem** is now “solved”
- **Fixed partitioning** results in **internal fragmentation**:
  - An **exact match** for the process and may not always be **available**
  - The partition may **not be used completely**

Figure: Summary

Dynamic Partitioning

Context

- The **relocation problem** is now “solved”
- **Fixed partitioning** results in **internal fragmentation**:
  - An **exact match** for the process and may not always be **available**
  - The partition may **not be used completely**

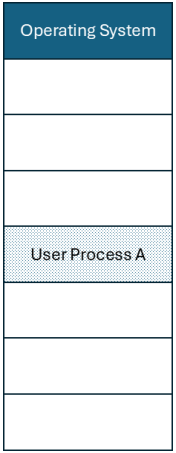


Figure: Summary

Dynamic Partitioning

Context

- The **relocation problem** is now “solved”
- **Fixed partitioning** results in **internal fragmentation**:
  - An **exact match** for the process and may not always be **available**
  - The partition may **not be used completely**

Figure: Summary

Dynamic Partitioning

Context

- The **relocation problem** is now “solved”
- **Fixed partitioning** results in **internal fragmentation**:
  - An **exact match** for the process and may not always be **available**
  - The partition may **not be used completely**

Figure: Summary

Dynamic Partitioning

Concept

- A process is allocated the **exact amount** of **contiguous** memory
- A **variable number of partitions**
- **Aims to reduce internal fragmentation**

Dynamic Partitioning

Example

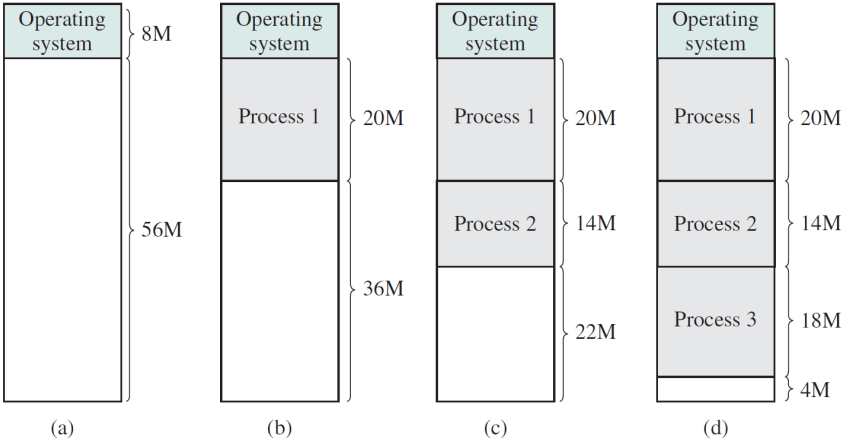


Figure: Dynamic partitioning (from Stallings)

Dynamic Partitioning

Swapping

- Swapping **moves processes** between the drive and main memory
- **Reasons** for swapping:
  - Some **processes** only **run occasionally**
  - The **total amount of memory** required **exceeds available memory**
  - **Memory requirements change** / cannot be **known in advance** (stack / heap)
  - We have **more processes** than **partitions** (assuming fixed partitions)
- Processes can be reloaded into a **different memory location** (base register changes)

Dynamic Partitioning

Reflection

- Fixed partitioning $\Rightarrow$ dynamic partitioning
  - **Made to measure** partitions result in improved **memory utilisation**
  - **Internal fragmentation** has been “solved”
- Any new issues?

Dynamic Partitioning

Swapping: Example

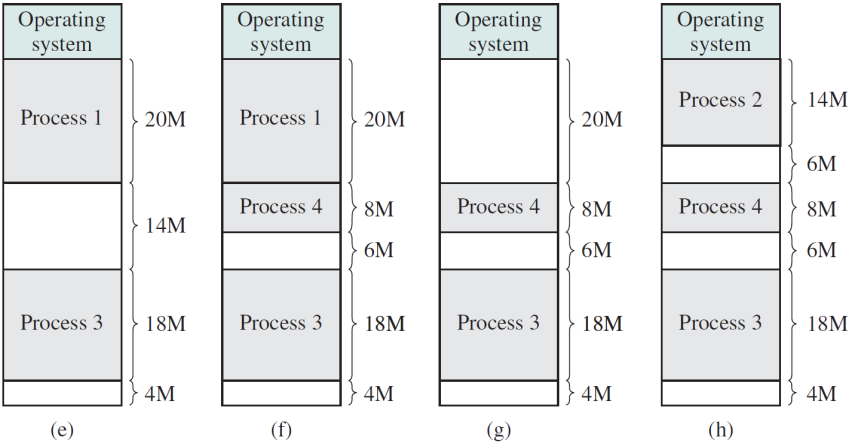


Figure: External fragmentation (from Stallings)

Dynamic Partitioning

Reflection

- Fixed partitioning $\Rightarrow$ dynamic partitioning
  - **Made to measure** partitions result in improved **memory utilisation**
  - **Internal fragmentation** has been “solved”
- Any new issues?

Dynamic Partitioning

Reflection

- Fixed partitioning $\Rightarrow$ dynamic partitioning
  - **Made to measure** partitions result in improved **memory utilisation**
  - **Internal fragmentation** has been “solved”
- Any new issues?
  - **External fragmentation** – small memory partitions distributed all over physical memory
    - **Swapping** a process out of memory will create “**a hole**” and is **time consuming**
    - A new process may not use the entire “hole”, leaving a small **unused block**
    - A new process may be **too large** for a given partition and the **overhead** of memory **compaction** to **recover holes** can be **prohibitive**
  - Tracking of **free space** and memory **allocation policies**

Dynamic Partitioning

Shortcomings

Figure: Memory layout (logical address space)

Dynamic Partitioning

Shortcomings

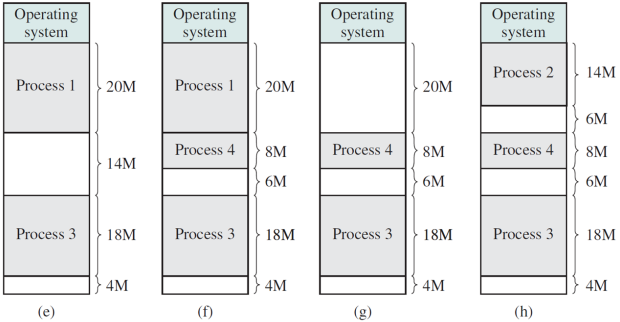


Figure: Fragmentation

Dynamic Partitioning

Shortcomings

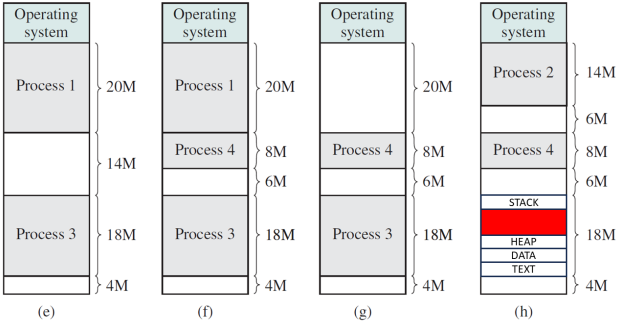


Figure: Fragmentation

Dynamic Partitioning

Reflection

- Fixed partitioning $\Rightarrow$ dynamic partitioning
  - **Made to measure** partitions result in improved **memory utilisation**
  - **Internal fragmentation** has been "solved"
- Any new issues?
  - **External fragmentation** – small memory partitions distributed all over physical memory
    - **Swapping** a process out of memory will create "**a hole**" and is **time consuming**
    - A new process may not use the entire "hole", leaving a small **unused block**
    - A new process may be **too large** for a given partition and the **overhead** of memory **compaction** to **recover holes** can be **prohibitive**
  - Tracking of **free space** and memory **allocation policies**

Dynamic Partitioning

Reflection

- Fixed partitioning $\Rightarrow$ dynamic partitioning
  - **Made to measure** partitions result in improved **memory utilisation**
  - **Internal fragmentation** has been "solved"
- Any new issues?
  - **External fragmentation** – small memory partitions distributed all over physical memory
    - **Swapping** a process out of memory will create "**a hole**" and is **time consuming**
    - A new process may not use the entire "hole", leaving a small **unused block**
    - A new process may be **too large** for a given partition and the **overhead** of memory **compaction** to **recover holes** can be **prohibitive**
  - Tracking of **free space** and memory **allocation policies**
  - Large "unused" **gaps in the logical address space**

Segmentation

Fine Grained Dynamic Partitioning

- **Dynamic partitioning** loads the **entire logical address space** into (**contiguous**) physical memory
  - Uses a **single base and bounds pair** per process
  - Results in **external fragmentation**
  - **Unused space** between stack and heap **takes up physical memory**
- **Segmentation** loads only the **relevant sections** into memory
  - Splits the logical address space into **separate contiguous segments**
  - Each segment is **loaded separately** in (contiguous) memory
  - Each segment has a different **base** and **bound** pair
  - The base and bound pair are stored in a the **segmentation table**
  - **Part of the logical address** is used as an **index** into the segmentation table

Dynamic Partitioning

Shortcomings

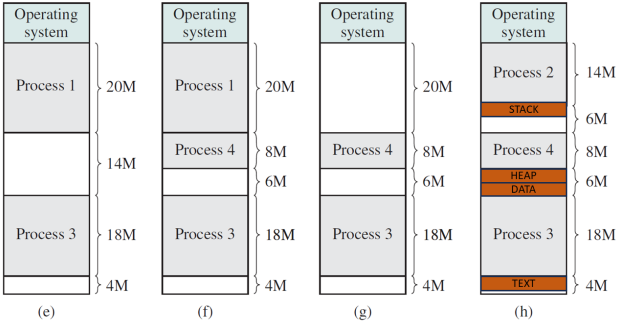


Figure: Segmentation

Segmentation

Segmentation Table

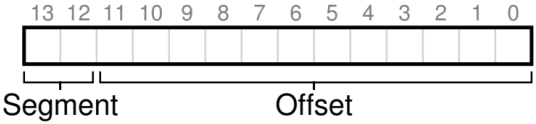


Figure: Logical address for segmentation (offset = position relative to start of segment)

Segment	index	Base	Bound	RWX
Code	00	...	...	...
Data	01	...	...	...
Heap	10	...	...	...
Stack	11	...	...	...

Segmentation

Segmentation Table & Challenges

- Segments can:
  - Have **protection bits** associated with them (RWX)
  - Be **shared** between processes (e.g. code segments)
- **MMU** must use the correct base / bound
- OS remains responsible for finding a sufficiently large **contiguous range of physical memory** for each segment (segments can be **fine-grained**)

Summary

Take Home Message

- **Contiguous** memory schemes: mono-programming, static and **dynamic partitioning**
- Principles of **relocation** and **protection** underpin paging and virtual memory!
- Internal and external **fragmentation**
- **Segmentation** and **segmentation tables**